Predicting Spotify Song Popularity Using Neural Networks

MTH 4320 Deep Learning

Fall 2026

Florida Institute of Technology

Melbourne, FL

Dr. Ryan White

October 4th, 2026

Lilly Davies (904027105), Brieleigh Wescott (904028067)

Table of Contents

1 Introduction **3**

1.1 Project Question 3

1.2 Project Goals 3

2 Dataset **3**

2.1 Data Source 3

2.2 Variable 4

2.3 Data Preprocessing 5

2.4 Exploratory Analysis 6

2.5 Feature Selection 8

2.6 Training/ Validation/ Test Split 9

2.7 Feature Standardization 9

3 Baseline Model: Linear Regression **10**

3.1 Why is a Baseline Model Needed? 10

3.2 Linear Regression Model 10

3.3 Baseline Results and Interpretation 10

3.3.1 Baseline Results 10

3.3.2 Baseline Interpretation 11

4 Neural Network Development **13**

4.1 Initial MLP Architecture 14

4.2 Network Width Experiment 14

4.3 Network Depth Experiment 15

4.4 Activation Function Experiment 15

4.5 Learning Rate Experiment 16

4.6 Batch Size Experiment 16

4.7 L2 Regularization 17

4.8 Number of Epochs 18

4.9 Early Stopping 18

4.10 Ablation Experiment 20

4.11 Final Neural Network 20

5 Final Model Results and Analysis **21**

5.1 Final Test Performance 21

5.2 Actual vs Predicted Popularity 23

5.3 Error Analysis 24

5.4 What Does This Reveal? 25

6 Discussion **26**

6. 1 What Was Learned? 26

6.2 Linear Regression vs. Neural Network 26

6.3 Key Modeling Decisions 26

6.4 Unsuccessful Experiments 27

7 Limitations **28**

8 Conclusion **28**

Appendix **29**

Appendix A: Team Contributions 29

1 Introduction

1.1 Project Question

Spotify quantifies song popularity on a scale of 0 to 100 based on its total plays and how recent they are, with more recent plays having a higher weight. Spotify also analyzes songs to create individual profiles that include each track's qualitative and quantitative audio features. Using these calculated metrics, this project will attempt to answer the question: Can a song's popularity on Spotify be predicted using its audio features?

1.2 Project Goals

The primary goal of this project is to utilize the Spotify-identified audio features to train a linear regression baseline model and a multilayer perceptron (MLP) neural network to predict a song's popularity. The linear regression baseline performance will be compared to the MLP network's performance through these quantified metrics: mean squared error (MSE), and mean absolute error (MAE). The comparison of these performance metrics will reveal the complexity of the relationship between audio features and popularity, as well as which model performs best on unseen data.

2 Dataset

2.1 Data Source

In order to find a dataset for the experiment, the website Kaggle was used. Kaggle is an online platform for data science and machine learning that is filled with millions of datasets, codes, and more. On this website, the dataset: Spotify Tracks Dataset | Audio Features was selected. This dataset contains information on approximately 114,000 Spotify tracks spanning 114 different music genres. Each observation is representative of one song and includes its own specific id, many different audio characteristics that can be used to describe the song, and a Spotify Popularity score. In short, this dataset has everything needed to be able to answer the initial question: can a song's popularity on Spotify be predicted using its audio features?

2.2 Variable

The original dataset has about 20 variables, not all of which are relevant to what is planned to be modeled. The response variable for this project is popularity, which is measured on a scale of 0 to 100 (a higher value indicates a more popular song). According to the description of the dataset, it states that the audio features were obtained using data associated with Spotify and its Web API. Spotify's audio features are numerical measurements rather than categorical. This makes this specific dataset perfect for a regression problem, because it can provide quantitative characteristics that can be compared to the response variable, which is also numerical! Because this project focuses on predicting song popularity rather than developing the Spotify audio features themselves, the methods used by Spotify to calculate these features will not be examined in detail.

For the initial modeling, seven quantitative song characteristics were chosen as predictor variables, and the popularity score, which is the response variable.

Model Variables and Descriptions

Variable	**Description**	**Range**	**Role**
duration_ms	Length of track (measured in milliseconds)	> 0ms	Predictor
danceability	Measures how suitable a track is for dancing based on its musical characteristics.	0.0-1.0	Predictor
energy	Measures the perceived intensity and activity of a track	0.0-1.0	Predictor
loudness	Overall loudness of a track (measured in decibels)	Typically -60db (extremely quiet) to 0dB (extremely loud)	Predictor

speechiness	Measures the presence of spoken words in a track	0.0-1.0	Predictor
valence	Measures the musical positivity conveyed by a track	0-1.0	Predictor
tempo	Estimated speed of the track (measured in beats per minute)	> 0 BPM	Predictor
popularity	Spotify's popularity score	0 (not popular) - 100(extremely popular)	Response

2.3 Data Preprocessing

Before beginning the exploratory analysis and developing the model, it is important to become intimate with the data by inspecting and cleaning it. First, the data was checked for missing values, where only three were found: one in artists, one in album_name, and one in track_name. Since none of these variables are being used in the model, the dataset did not need to be treated. One surprising part of this dataset was the number of duplicate tracks. The dataset contained only 450 completely identical rows, but had 24,259 duplicate track IDs. This is a significant percentage of the original 114,000 tracks that were present in the beginning. To address this issue, duplicates were removed based on track_id, keeping only one observation per unique song. This left 89,741 unique tracks, and a final check confirmed that there are no remaining duplicate ID tracks.

Finally, eight variables were isolated to be used for the analysis: duration_ms, danceability, energy, loudness, speechiness, valence, tempo, and popularity. These variables were then triple checked to guarantee there were no missing values. One additional song, titled 'nan,' contained values of 0 for all variables and was removed. The final cleaned dataset had 89,740 tracks with complete data for every variable used in the model.

2.4 Exploratory Analysis

After cleaning the dataset, an exploratory analysis was performed to better understand the distributions of the variables and their relationships not only with popularity but also with one another. Below are four visualizations to better describe these relationships.

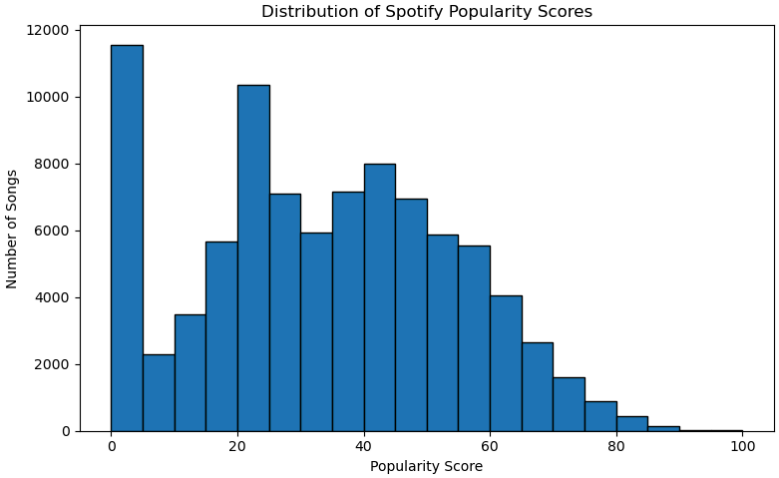


Figure 1: Distribution of Spotify Popularity Scores

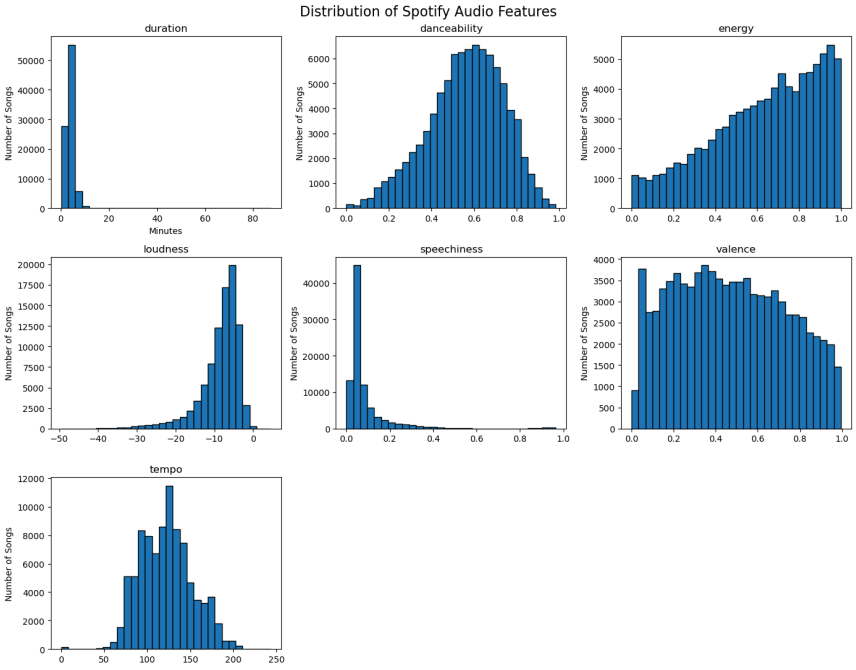


Figure 2: Distribution of Spotify Audio Features

The distribution graphs showed that the data is not evenly distributed. An important observation is that there are far fewer songs with very high popularity scores. This is important for the model because it will have fewer highly popular songs to learn from and may have more difficulty predicting songs at the end of the popularity scale rather than the beginning.

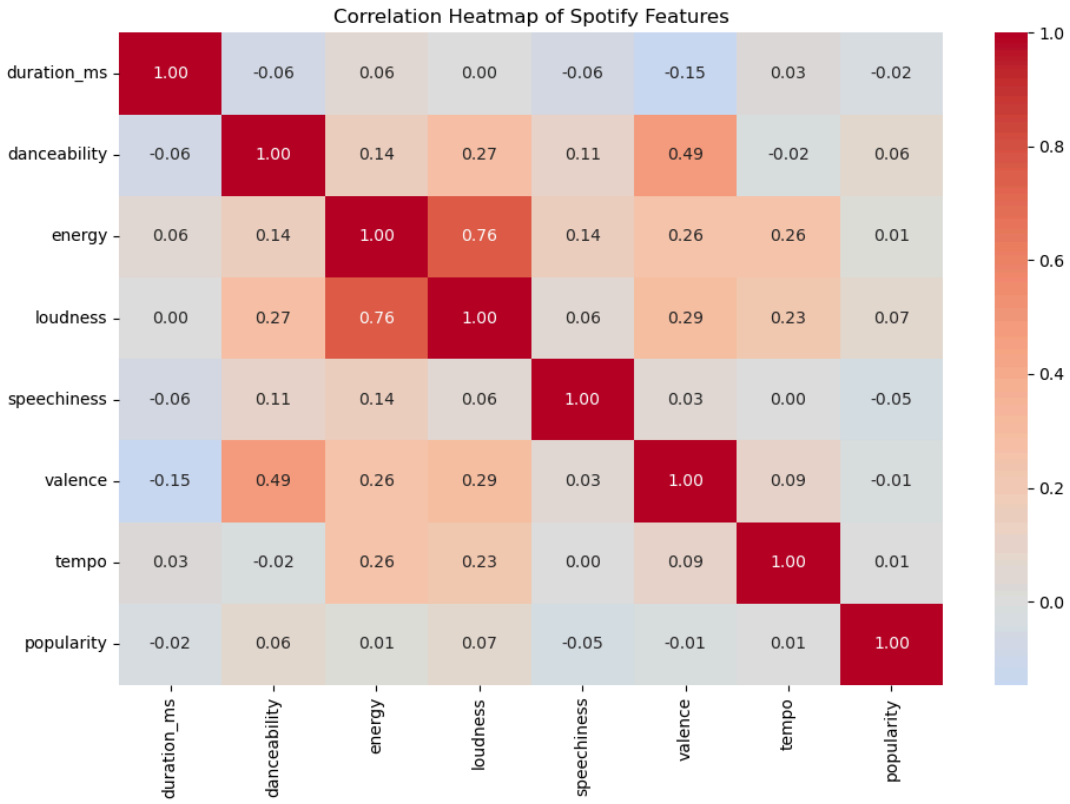


Figure 3: Correlation Heatmap

The correlation heatmap showed that none of the individual features seems to have a strong linear relationship with popularity. The largest correlation was only 0.07 for loudness and 0.06 for danceability. It was also found that some predictors are largely related to each other, like energy and loudness (correlation of 0.76), which makes sense as these are logically related to each other. This uses Pearson correlation; Spearman was also tried, but the results were almost identical.

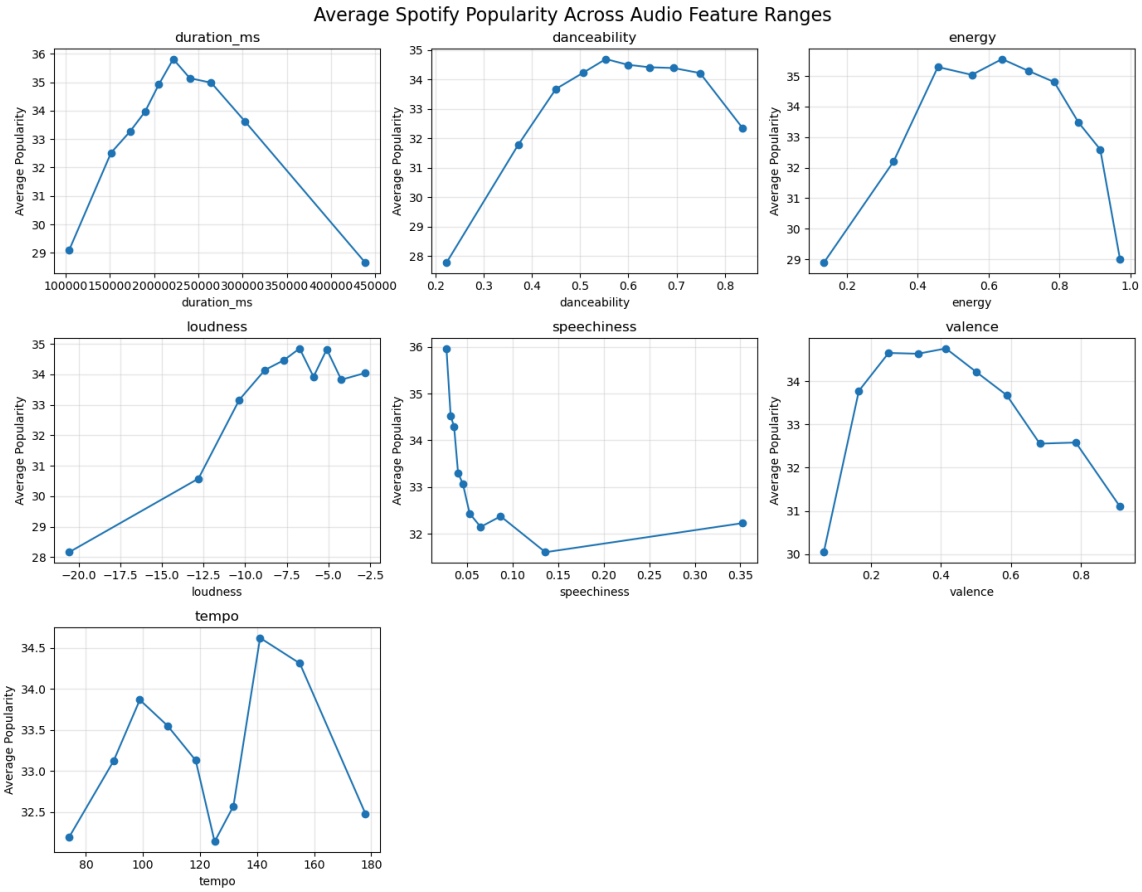


Figure 4: Average Spotify Popularity Across Audio Features Ranges

The above graphs show that there are still patterns in this data. Several features have a curved relationship with popularity rather than being very scattered or a simple straight line. This means that low correlation may not necessarily mean that the feature is useless for prediction.

2.5 Feature Selection

Based on the exploratory analysis, all seven original audio features were kept. Even though none of the features had a strong, if any, linear correlation with popularity, the graphs showed that they may have a nonlinear relationship that could still be useful for the prediction. Therefore, removing features based only on their correlation with popularity could cause useful information to be lost.

One relationship that received particular attention was energy and loudness, which had a correlation of 0.76. This suggests that the two features contain some overlapping information. However, energy was not removed right away because correlation does not mean the two variables provide exactly the same information to the model. Instead, the models will be tested with both, and the results will be compared.

For the initial models, all seven features will be included. Since none of the features showed a strong correlation with popularity, there was no clear feature that stood out as one that should be removed. Although this was not exactly what was hoped to be found, it means all seven features will be kept, allowing the models to determine whether combining them can uncover patterns that were not obvious from correlation alone.

2.6 Training/ Validation/ Test Split

Before building the models, the cleaned dataset was split into training, validation, and testing sets using a 70/15/15 split. This means 70% of the data is used to train the models, 15% is used to compare and tune the models, and the final 15% is saved for testing. A random seed of 4540 was used so that the same songs are placed into each group every time the code is run. This allows the results to be reproduced and the testing of the model to remain constant. This will be done when running the models and is not featured in the CSV file.

2.7 Feature Standardization

Before training the models, the seven predictor variables will be standardized using the StandardScaler function. This is helpful because the features are measured on very different scales. For example, danceability ranges from 0 to 1, while duration is measured in thousands of milliseconds. Standardization puts the features on a similar scale so that variables with larger numbers do not have an unfair influence on the model. This will only be done based on the training data. Doing this after the data split prevents information from the validation or test sets from leaking into the training process. The popularity score will not be standardized because it is the response variable, and the final predictions should remain on Spotify's original 0–100 popularity scale.

3 Baseline Model: Linear Regression

3.1 Why is a Baseline Model Needed?

A baseline model acts as the ‘control group’ when analyzing data. For this project, the linear regression provides a comparison for the MLP model’s output performance to determine whether its nonlinear layers and complexity are necessary.

3.2 Linear Regression Model

When predicting an output that has a linear relationship to its inputs, a linear regression model is used. It calculates a weight for each input that relates to how strongly the input affects the change to the predicted output. These weight terms are multiplied by each input and summed together to create the ‘linear’ form of this formula. A bias or intercept term is also learned and included in this summation to shift the predictive line or plane.

3.3 Baseline Results and Interpretation

3.3.1 Baseline Results

After fitting the model using the training data, then using this model to predict the validation outputs, each validation MSE was compared to select the best performing hyperparameters. These hyperparameters were then used to analyze popularity predictions on the training, validation, and test sets. From here, the performance metrics were calculated to analyze the model’s fit to the data. Each data set was standardized based on the mean and standard deviation of the training set, as the range of each feature was drastically different. This standardization prevented overflow or domination of large numeric features compared to smaller numeric features.

Best Found Hyperparameters

Learning Rate	0.001
Max Iterations	5000
Tolerance	0.001

Final Metrics Using Best Found Hyperparameters

Training MSE	417.15236
Training MAE	16.929836
Training R^2	0.0177186
Validation MSE	413.86879
Validation MAE	16.816058
Validation R^2	0.0160012
Test MSE	415.55432
Test MAE	16.906551
Test R^2	0.0135982

3.3.2 Baseline Interpretation

The results of the linear model's performance reports that MSE, MAE, and R^2 metrics are very similar across sets, showing that the model performs consistently across training, validation, and testing. However, the MSE and MAE values are relatively high. The MAE explains the error most clearly, showing that the popularity predictions are off by approximately 17 points on average. As the range of popularity is only 0-100, this is nearly ⅙ of the entire range. An R^2 of approximately 0.015 shows that the linear model only explains about 1.5% of the variation in popularity.

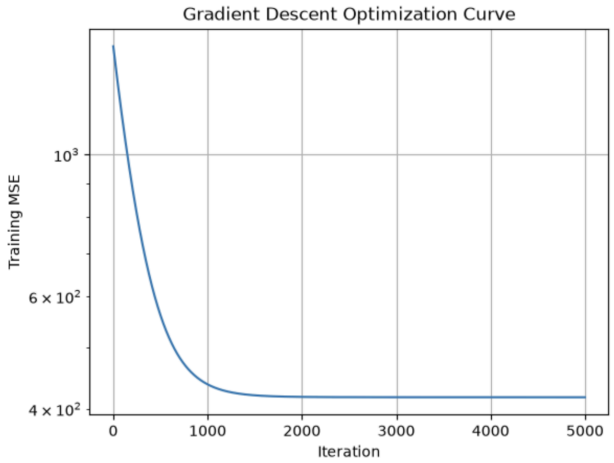


Figure 5: Gradient Descent Optimization Curve

The Gradient Descent Optimization Curve shows that the model does converge after about 1200 iterations, though this does not mean that it predicts well.

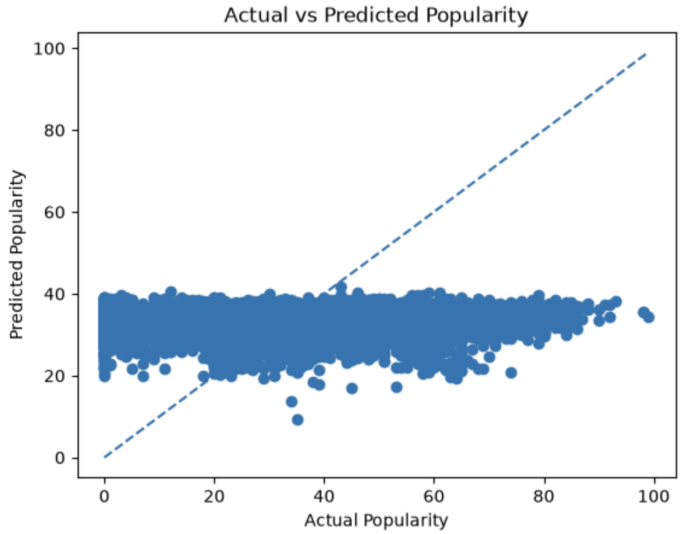


Figure 6: Actual vs Predicted Popularity

The Actual vs Predicted Popularity graph shows that the model does not predict popularity over about 40. This is most likely due to the model predicting popularity around the average popularity. This would occur due to a poor linear relationship between the inputs and the output, where the model would have little to no predictive power, determining very popular songs from unpopular songs.

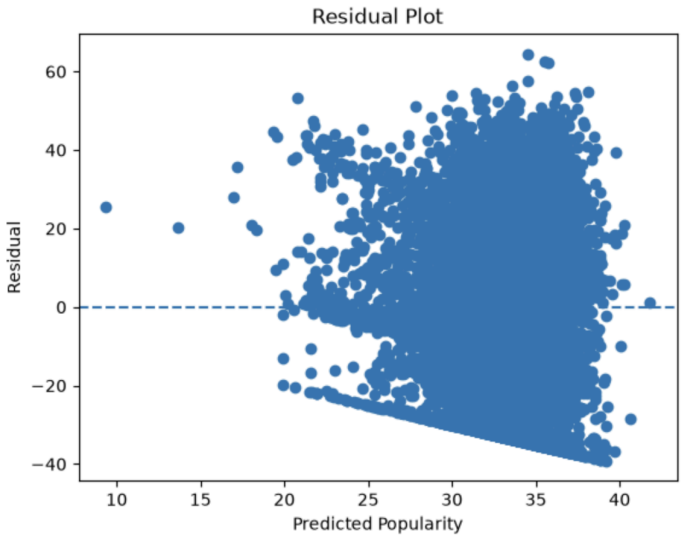


Figure 7: Residual Plot

The residual plot shows the difference between the actual popularity and the predicted for each song. For a good predictive model, this value should be very close to 0 for each song; however, for this model, it does not show this shape and instead forms a more vertical than horizontal shape.

Upon analysis of all 3 metrics and the graphics, the data clearly shows that the Spotify audio features have a poor linear relationship with a song's popularity. Additionally, the linear model does not perform well at predicting popularity.

4 Neural Network Development

After giving linear regression a chance to predict song popularity, a neural network was used to see if a more flexible model could do a better job. The neural network used the same seven standardized features and used those seven features to, again, predict a song's popularity score. Rather than just randomly throwing different settings at the model and hoping it would work, the model was tuned step by step, one piece at a time. Training and validation MSE were used to compare models, while the test data remained untouched until the very end.

4.1 Initial MLP Architecture

The model started pretty simple with a 7-16-1 architecture. This means there were seven input features, one hidden layer with 16 neurons, and one output neuron for predicted popularity. The initial model used a sigmoid activation function, MSE loss, and a learning rate of 0.01. This first attempt gave a training MSE of 412.83 and a validation MSE of 411.56. These results are not amazing, but also not horrible. Most importantly, this gave a starting point. From there, different settings began to be tested.

4.2 Network Width Experiment

First, the number of neurons that should be in the hidden layer was tested. The model was tested with 8, 16, 32, and 64 neurons while keeping all other aspects of the model constant.

Network Width Results

Hidden Neurons	**Training MSE**	**Validation MSE**
8	413.25	412.16
16	**412.83**	**411.56**
32	414.40	413.10
64	415.94	414.84

The winner was 16 neurons, with a validation MSE of 411.56. It was rather interesting that making the network bigger actually made it worse. Apparently, more neurons don't mean more intelligence in this case. Since 16 performed the best, this is what will be kept moving forward.

4.3 Network Depth Experiment

Next came the most logical follow-up question: if widening the layer did not help, should it be made deeper instead? To answer this question, networks with one, two, and three hidden layers were compared. Keeping that 16 neurons per hidden layer.

Hidden Layer Results

Hidden Layers	Training MSE	Validation MSE
1	**412.83**	**411.56**
2	419.33	418.90
3	424.10	423.44

The answer to the question was a pretty solid no. One hidden layer had the lowest validation MSE at 411.56. Increasing the layers seemed to increase the MSE. The neural network apparently preferred to keep things simple, so the 7-16-1 architecture remained.

4.4 Activation Function Experiment

At this point, the MSE values were exactly where they started, as it seemed that changing the network size was not getting very far at all. So the activation function was tested next. Sigmoid, ReLU, and Tanh were tested.

Activation Function Results

Activation	Training MSE	Validation MSE
Sigmoid	412.83	411.56
ReLU	411.95	410.93
Tanh	**397.33**	**394.40**

Finally, something exciting happened! ReLU gave a small improvement, but Tanh dropped the validation MSE all the way down to 394.40. This makes Tanh the obvious choice moving forward.

4.5 Learning Rate Experiment

With Tanh officially joining the model, learning rates of 0.001, 0.005, 0.01, and 0.05 were then tested.

Learning Rate Results

Learning Rate	**Training MSE**	**Validation MSE**
0.001	442.23	442.32
0.005	406.34	404.84
0.010	397.33	394.40
0.050	**393.74**	**390.69**

The learning rate of 0.001 was not only painfully slow, but produced by far the worst results. It seems that performance improved as the learning rate increased, with 0.05 producing the best validation MSE of 390.69. So, among the learning rates tested, 0.05 was the clear choice, reducing the MSE from the original 0.010 that was used.

4.6 Batch Size Experiment

Mini-batch training was used on this model. Mini-batch training means the neural network does not look up all the songs at once when updating itself; instead, it breaks the training data into smaller batches. Next comes the batch size. Batch sizes of 32, 64, 128, and 256 were tested to see how much data the network should process before updating its weights.

Batch Size Results

Batch Size	Training MSE	Validation MSE
32	434.08	430.69
64	417.81	413.52
128	**396.70**	395.26
256	397.41	**394.84**

The smaller batch sizes did not perform as well as the larger ones. Now, batch size 128 technically had the lowest training MSE, but 256 had the lowest validation MSE at 394.84. Since performance on the data the model was not trained on is more important, 256 was chosen to be the batch size.

4.7 L2 Regularization

L2 regularization was then introduced. L2 regularization is a machine learning technique used to prevent overfitting by adding a penalty proportional to the square of the model's weights to the loss function.

L2 Regularization Results

L2	Training MSE	Validation MSE
0	397.41	394.84
0.0001	**394.04**	**391.89**
0.001	394.54	392.17
0.01	397.79	395.74

A small amount of regularization helped. Setting L2 to 0.0001 gave the lowest validation MSE of 391.89. However, more regularization was definitely not better. By the time L2 reached 0.01, validation MSE increased to 395.74. L2 = 0.0001 was kept.

4.8 Number of Epochs

Next, the length of time the model should train was tested. Models were trained for 5, 10, 20, 50, and 100 epochs.

Epoch Results

Epochs	Training MSE	Validation MSE
5	**393.91**	**391.25**
10	397.06	395.22
20	394.04	391.89
50	397.14	396.89
100	395.52	394.65

This experiment was a little chaotic. Training longer did not make the model consistently better. Five epochs actually had the lowest validation MSE of the fixed values tested, while 50 and 100 epochs performed the worst. Because mini-batch training shuffles the observations and updates the model repeatedly, the MSE can bounce around from epoch to epoch. Picking an arbitrary stopping point was clearly not ideal. Instead of trying to guess exactly when the model should stop, the validation data was used to determine the stopping point. This is where early stopping comes in.

4.9 Early Stopping

Early stopping allowed the model to train for up to 100 epochs, but it kept track of validation MSE along the way. Whenever validation performance improved, that version of the

model was saved. If validation MSE failed to improve for five consecutive epochs, then the training stopped. In this case, the model was trained for 16 epochs, but its best performance occurred at epoch 11, where validation MSE reached 389.44. The weights from epoch 11 were then kept.

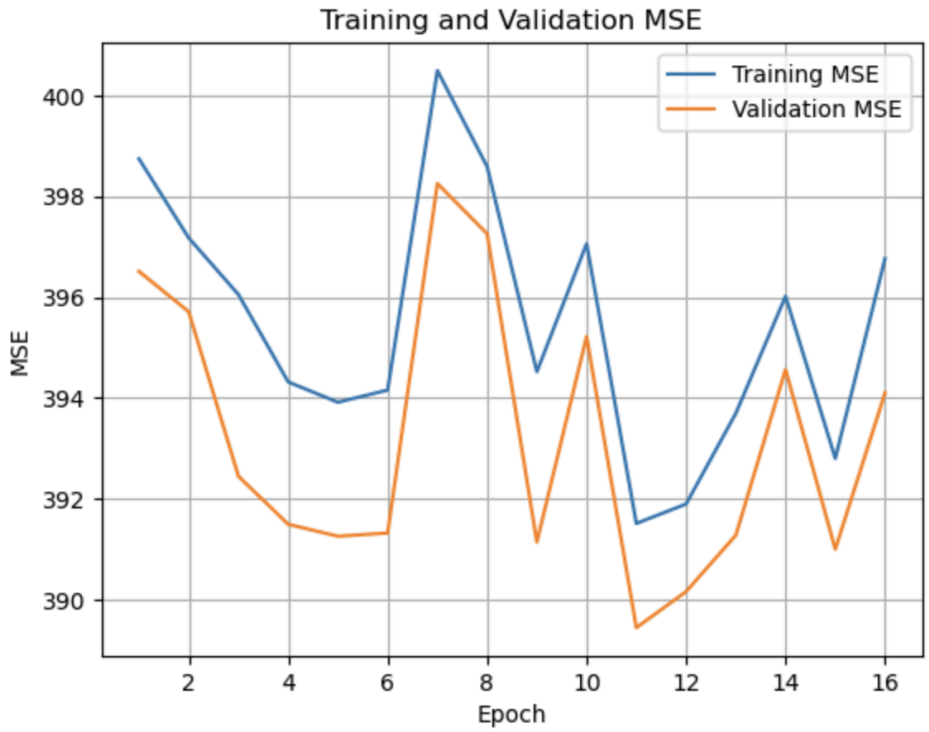


Figure 8: Training and Validation MSE

The graph is admittedly not the world's prettiest downward curve. Both training and validation MSE bounce around quite a bit. However, that is also exactly why early stopping was useful. Instead of assuming that more training automatically meant a better model, the model could be saved whenever it actually performed the best. Validation MSE reached its lowest point at epoch 11 and then began bouncing back upward. Early stopping therefore, prevented the model from simply training to 100 epochs and ending up with a worse model. The validation MSE stayed slightly lower than the training MSE throughout training, which suggests the model was not seriously overfitting. Instead, the main issue was that the MSE bounced around from epoch to epoch because of the mini-batch training.

4.10 Ablation Experiment

Finally, an ablation experiment was performed to make sure that L2 regularization and early stopping were actually helping instead of just being extra nonsense. These components were removed one at a time.

Ablation results

Model	**Validation MSE**
L2 + Early Stopping	**389.44**
Early Stopping Only	391.13
L2 Only	394.65
Neither	394.31

The full model with both L2 and early stopping won, producing the lowest validation MSE of 389.44. Removing L2 increased the MSE to 391.13, while removing early stopping increased it to 394.65. Interestingly, L2 by itself was actually slightly worse than using neither technique. So L2 was not as great on its own, but it is most useful when paired with early stopping. Overall, the ablation experiment gave evidence that keeping both L2 regularization and early stopping was the best choice for the final model.

4.11 Final Neural Network

After all of the experimenting, there was finally a winner. The final model used the settings below:

Final Neural Network

Setting	**Final Choice**
Architecture	7 → 16 → 1
Activation	Tanh
Learning Rate	0.05
Batch Size	256
L2	0.0001
Early Stopping	Patience = 5
Best Epoch	11
Validation MSE	389.44

After testing just about everything that could be thought of, the best model ended up being relatively simple. It used only one hidden layer with 16 neurons, but the right activation function and training settings made a noticeable difference. The final validation MSE of 389.44 was the lowest of all the models tested.

5 Final Model Results and Analysis

After all of the experimenting and tuning in Section 4, it was finally time to introduce the neural network to the test set. The test data had been completely left alone during model development, so these results show how well the final model performed on songs it had never seen before.

5.1 Final Test Performance

The table below summarizes the progression from the original linear regression model to the final neural network. For each neural network experiment, the setting with the lowest validation MSE is shown.

Model/ Experiment	**Best Setting**	**Training MSE**	**Validation MSE**
Initial MLP	7 → 16 → 1	412.83	411.56
Width	16 neurons	412.83	411.56
Depth	1 hidden layer	412.83	411.56
Activation	Tanh	397.33	394.40
Learning Rate	0.05	393.74	390.69
Batch Size	256	397.41	394.84
L2 Regularization	0.0001	394.04	391.89
Early Stopping	Epoch 11	–	389.44

After selecting the final model, it was evaluated once on the untouched test set.

Metric	**Linear Regression**	**Final Neural Network**
MSE	415.55	390.66
RMSE	20.38	19.77
MAE	16.91	16.00

The neural network performed better than linear regression across all three error measurements. The MAE is probably the easiest to understand: on average, the predicted popularity score was about 16 points away from the actual popularity score. The improvement

over linear regression was not enormous, but it showed that allowing the model to learn nonlinear relationships between the song features helped improve its predictions.

5.2 Actual vs Predicted Popularity

To get a better idea of how the model was actually making predictions, the predicted popularity of each song was plotted against its actual popularity. The dashed line represents a perfect prediction, where the predicted and actual popularity scores would be exactly the same.

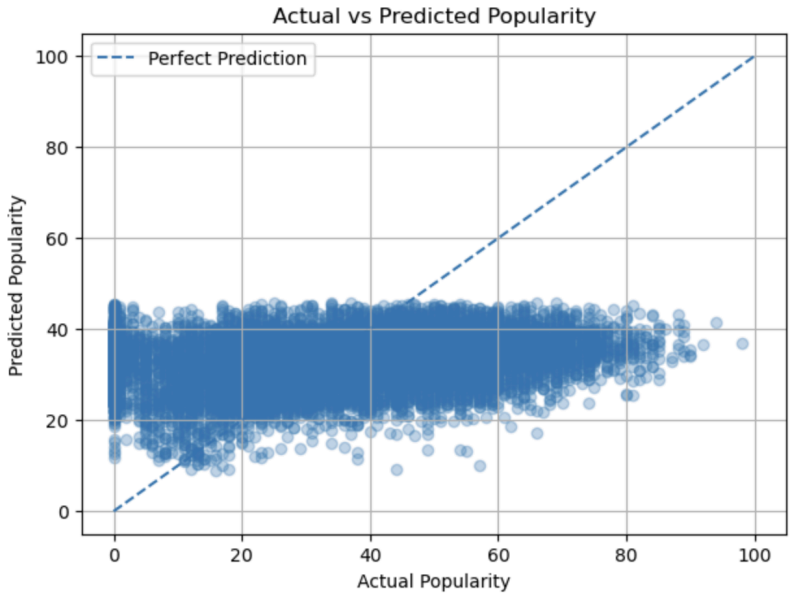


Figure 9: Actual vs Predicted Popularity Neural Network

One thing immediately stands out: the neural network really likes the middle. Most of its predictions fall between about 20 and 45, even though the actual popularity scores range from 0 to almost 100. Because of this, the model tends to overpredict songs with very low popularity and underpredict songs with very high popularity. For example, songs with actual popularity scores above 60 are almost always below the perfect-prediction line. The model recognizes some differences between songs, but it struggles to make extreme predictions at either end of the popularity scale. This helps explain why the MAE remained around 16 points even after all of the tuning. The model became better than the linear regression baseline, but it still tended to play it safe and predict somewhere near the middle rather than predicting extremely low or extremely high popularity scores.

5.3 Error Analysis

The scatterplot showed that the model really liked predicting somewhere in the middle, so the effect this had on songs at different popularity levels was examined. The test songs were divided into five groups based on their actual popularity, and the MAE and average error were calculated for each group. MAE tells how far off the predictions were, while average error tells the direction of the mistakes. A positive average error means the model tended to predict too high, while a negative value means it tended to predict too low.

Popularity Range	Songs	MAE	Average Error
0-20	3691	23.17	+23.11
21-40	4588	6.42	+2.92
41-60	3887	14.85	-14.85
61-80	1216	31.99	-31.99
81-100	79	48.15	-48.15

The model performed best on songs with popularity scores between 21 and 40, where the MAE was only 6.42. This was also the only group where the MAE and average error were noticeably different. A MAE of 6.42 means the predictions were about six points away from the actual score on average, while the average error of only +2.92 means that some predictions were too high and others were too low. Those positive and negative errors partially canceled each other out. For the other popularity ranges, the MAE and average error were almost identical in size because the model was making mistakes almost entirely in one direction. Songs between 0 and 20 were almost always overpredicted, while songs between 41 and 60 were underpredicted. This confirms what was seen in the scatterplot: the model really likes the middle. It pulls low-popularity songs upward and high-popularity songs downward instead of making predictions near the extremes.

Interestingly, across the entire test set, the average error was only −0.12. The model overpredicted 6,706 songs and underpredicted 6,755 songs, which sounds almost perfectly balanced. However, that does not mean the predictions were almost perfect. The positive errors

on unpopular songs and negative errors on popular songs were simply canceling each other out. This is why the overall MAE of 16.00 gives a much better picture of how far away the predictions actually were.

5.4 What Does This Reveal?

So, can a song's popularity be predicted based only on its audio features? The answer to that is: not very well.

Although the neural network performed better than the linear regression model, the improvement was fairly small. The final model still had an MAE of 16.00, meaning its predictions were off by about 16 popularity points on average. More importantly, the error analysis showed that the model was not actually learning to predict the full range of popularity very well.

The model was best at predicting songs in the 21–40 range and struggled badly everywhere else. It overpredicted unpopular songs, underpredicted popular songs, and was especially bad at predicting extremely popular songs. For songs with popularity scores between 81 and 100, the MAE reached 48.15 points. At that point, the model is not just a little off, but it is missing a huge part of what makes those songs different. Overall, the model seems to have learned that predicting somewhere in the middle is usually the safest option. All of the hyperparameter tuning improved the model, but no combination of neurons, learning rates, batch sizes, regularization, or early stopping fixed this fundamental problem.

This suggests that the seven audio features simply do not contain enough information to accurately predict Spotify popularity. Popularity depends on much more than what a song sounds like. The model knows the tempo and danceability of a song, but it has no idea who released it, how much it was promoted, whether it was added to major playlists, or whether it suddenly went viral. So while the neural network technically improved on the baseline, the larger takeaway from this project is that audio features alone are not enough to make strong popularity predictions. In this case, figuring out what doesn't work ended up being one of the more useful results of the project.

6 Discussion

6. 1 What Was Learned?

Throughout the entirety of the training, validating, and testing process, it was determined that a song's Spotify calculated popularity score is not predicted well using its Spotify-identified audio features. This applied to both the linear and neural models despite adjusting several experimental parameters. However, the neural model did perform slightly better, showing evidence of some nonlinear relationships between the audio features and the popularity score of a song. These results confirm the findings of the initial exploratory analysis, which showed no initial evidence of a linear relationship but the potential for some nonlinear patterns.

6.2 Linear Regression vs. Neural Network

Although neither model had a very successful predictive capacity toward the popularity metric, the neural network did perform slightly better across the MSE, RMSE, and MAE metrics. This difference is best described by the MAE, where the linear model predicted popularity was different from the actual by 16.9 points on average, and the MLP's predicted popularity was off from the actual by 16 points on average. This is not a major difference in performance, but shows the evidence of some nonlinear relationships that slightly improved the MLP's predictive ability.

Both models demonstrated the tendency to predict popularity between approximately 20-40 points, which falls around the mean of the data's popularity overall. This is due to the model's inability to predict based on the weak relationships, so they did not make predictions at the extreme ends of the 0-100 range. They also both showed only small improvements when adjusting hyperparameters, which demonstrated that tuning the models could not create a relationship between features and popularity where it did not initially exist.

6.3 Key Modeling Decisions

Before creating either model, the first primary steps were identifying which audio features to use, splitting them into sets, and then standardizing them. The 70/15/15 split was

determined based upon typical model training practices, then standardization was done based upon the mean and standard deviation of the training data. The key error metrics were chosen to be MSE and MAE as they expose large errors and provide easily interpretable metrics for the difference in predicted vs actual performance.

For the linear model, the key hyperparameters were chosen to be the learning rate, iterations, and tolerance, as these are the most impactful for a baseline linear regression model. Three of each type were tested to determine the best configuration for optimizing convergence and training performance. Once these were tested on the validation data to determine performance on unseen data, the best of the 3 were used on the test set to calculate the final MSE, MAE, and R^2 values, showing the best found linear regression model.

As the linear model did not perform well, it was important to thoroughly explore the capability of the neural model to truly determine if there were relationships between the audio features and popularity. Network width and depth, key hyperparameters, and training length were the crucial elements of this experiment. Through the in-depth evaluation of each of these parameters, it was found that a simpler network performed slightly better. Tanh was the most beneficial activation function in determining nonlinear relationships. A learning rate of 0.05 provided the optimal balance between convergence and learning stability. A batch size of 256 was the best balance for stability of updating weights per epoch. Training with multiple epochs only increased performance when there were fewer than 10. An L2 regularization value of 0.0001 was beneficial in reducing overfitting, but was only significant when paired with early stopping. However, it is important to note that even though these parameters optimized the neural network, they did not significantly improve the model's predictive capabilities.

6.4 Unsuccessful Experiments

During linear regression training, it was found that the model did not converge when the tested learning rates were [1e-6, 1e-7, 1e-8], which produced an R^2 of -2. These had to be reduced to [1e-2, 1e-3, 1e-4] for convergence. During the neural network training, there was an attempt to change the weights between training, validation, and test sets to 60/20/20, but it did not provide a negligible difference to the outcome, so this experiment was dropped.

7 Limitations

Considering the Kaggle dataset was composed of strictly Spotify-identified audio features, there was no exploration of other factors that could affect a song's popularity. A song's audio factors alone do not create a strong enough influence on popularity that others, such as artist popularity, marketability, cultural trends, and genre, might. The Kaggle dataset also did not provide any explanation as to how these audio features and popularity scores were calculated. As such, it was assumed that the data accurately represented each song. Any measurement or calculation errors by Spotify and any notation errors by Kaggle, therefore, are not accounted for and may have impacted the true relationships. Another limitation was the spread of the popularity, which was not evenly distributed from 0 to 100. In fact, most of the data was in the lower half of this range, which enabled the models to favor these popularity scores due to an imbalance in representation.

8 Conclusion

In this exploration of the relationship between audio features and song popularity, the linear regression was the control. The comparison between this control and the neural network showed that the increase of complexity did not significantly improve the ability to predict song popularity. However, the neural network did identify small nonlinear relationships that the linear regression could not, improving the final test MSE by approximately 6% and the MAE by 5.4% or 0.91 popularity points. The final performance of both models strongly suggests that song popularity cannot be predicted based on audio features alone.

To expand upon this topic, future research can be done to identify features that more strongly impact song popularity. Some possible metrics may be artist popularity, release schedules, social media exposure, and song genre. Additionally, datasets from a different source than Kaggle or Spotify could be tested to validate the claim that audio features are not good predictors of song popularity. In any case, for future research, this topic holds strong practical value for both artists and marketing purposes, for what metrics most strongly influence song popularity.

Appendix

Appendix A: Team Contributions

Brieleigh Wescott

- Dataset preparation, cleaning, and exploratory analysis
- Feature selection, data splitting, and standardization
- Neural network development and implementation
- Neural network hyperparameter tuning and controlled experiments
- Regularization, early stopping, and ablation experiments
- Final neural network evaluation and error analysis
- Writing of Sections 2, 4, and 5

Lilly Davies

- Project introduction and problem definition
- Linear regression baseline development and implementation
- Linear regression tuning, evaluation, and interpretation
- Overall model discussion and comparison
- Project limitations and conclusion
- Writing of Sections 1, 3, 6, 7, and 8